

JavaScript Essencial para Iniciantes

Material de apoio didático para estudantes e instrutores de programação web front-end.

1. Introdução à Linguagem e Execução

JavaScript é a linguagem de programação padrão da web para interatividade do lado do cliente (client-side). Enquanto o **HTML** constrói a estrutura básica e o **CSS** define a apresentação visual, o **JavaScript** orquestra comportamentos, lógica e resposta a eventos gerados pelo usuário.

Onde o código é executado?

O navegador possui um motor interno (como o V8 no Chrome) que interpreta e executa scripts. Os testes podem ser feitos diretamente no Console do DevTools (tecla F12) ou linkando um arquivo na tag `<script>`.

2. Declaração de Variáveis: const, let e var

No padrão moderno (ECMAScript 6+), a declaração de variáveis é orientada à previsibilidade e isolamento de escopo:

const (Padrão)

Para referências imutáveis. O valor atribuído não pode ser sobrescrito.

let (Mutável)

Para valores que mudam ao longo do tempo (contadores, acumuladores).

var (Legado)

Escopo global ou de função. Sujeito a bugs e desaconselhado em novos projetos.

```
const taxaFixa = 0.15; // Não pode ser alterado
let pontuacao = 100; // Pode ser reatribuído
pontuacao = 120; // Operação válida
```

3. Tipos Primitivos de Dados

Tipo	Descrição	Exemplo de Sintaxe
String	Cadeias de caracteres textuais delimitadas por aspas.	const nome = "Ana";
Number	Valores numéricos (inteiros e pontos flutuantes).	const preco = 49.90;
Boolean	Valores binários lógicos de controle.	const ativo = true;
Undefined / Null	Ausência de valor (não inicializado vs. intencional).	let vazio = null;

4. Operadores e Tomada de Decisão

JavaScript permite comparar variáveis através de operadores estritos, avaliando tipo e valor sem coerção implícita indesejada.

Comparação Estrita vs. Solta

```
// Sempre priorize === e !== 5 ===  
"5" // false (tipos diferentes) 5 == "5" //  
true (converte tipo - evite) 10 !== 20 //  
true
```

Estrutura Condicional Simples

```
const idade = 18; if (idade >= 18)  
{ console.log("Acesso liberado"); } else {  
  console.log("Bloqueado"); }
```

5. Funções Modernas: Arrow Functions

Funções empacotam blocos de lógica reutilizável. As *Arrow Functions* (`=>`) tornam o código mais sucinto e moderno:

```
// Função tradicional function somar(a, b) { return a + b; } // Arrow function moderna  
equivalente const somarArrow = (a, b) => a + b; const resultado = somarArrow(15, 25); //  
Retorna 40
```

6. Manipulação do DOM (Document Object Model)

O DOM é a representação do HTML em forma de árvore de objetos na memória do navegador. O JavaScript conecta a lógica à interface através de três operações fundamentais:

1. Seleção: `document.querySelector(seletor)`

Localiza elementos no documento utilizando seletores CSS (por tag, classe ou ID).

2. Escuta: `elemento.addEventListener(evento, callback)`

Detecta ações do usuário, como cliques do mouse ('click') ou digitação ('input').

3. Modificação: `elemento.textContent` e `elemento.style`

Atualiza o conteúdo de texto renderizado ou altera propriedades de estilo CSS inline.

```
const botao = document.querySelector("#meuBotao"); const texto =  
document.querySelector("#titulo"); botao.addEventListener("click", () => { texto.textContent  
= "Texto Atualizado!"; texto.style.color = "#2563eb"; });
```

Roteiro de Ensino para o Instrutor

Sugestões de fala, gestão do tempo e pontos críticos para a condução da aula.

Quadro Geral de Gestão do Tempo

Etapa	Duração	Foco do Professor e Alinhamento Pedagógico
Abertura	5 min	Apresentar o papel do JavaScript na tríade web. Desmistificar receios de lógica com exemplos reais de botões e telas que os alunos já usam.
Variáveis	12 min	Enfatizar o uso padrão de <code>const</code> e explicar por que o <code>var</code> caiu em desuso. Demonstrar erros comuns de reatribuição ao vivo.
Lógica & DOM	18 min	Demonstrar como o navegador lê o código do topo ao fundo e conectar o <code>querySelector</code> ao clique do mouse.
Mini-Projeto	25 min	Live coding guiado do contador interativo. Conduzir os alunos linha a linha, reforçando a separação entre estado e interface.

Pontos Críticos para Esclarecer aos Alunos

1. Posição da tag `<script>`

Explique por que inserir a tag antes do fechamento de `</body>` é importante: se o script rodar antes da leitura do HTML, o `querySelector` retornará `null`, gerando o erro comum *"Cannot read properties of null"*.

2. Diferença entre Variável (Memória) e DOM (Visual)

Iniciantes frequentemente esquecem de sincronizar as duas partes. Mostre que alterar uma variável como `contador++` apenas altera a memória do computador; é obrigatório escrever `visor.textContent = contador` para refletir na tela.

3. Notação de Ponto e Escopo de Evento

Reforce que `addEventListener` espera uma função (callback). Erro clássico: colocar parênteses na função chamada, o que dispara a execução imediata em vez de aguardar o clique.

Mini-Projeto: Contador Interativo

Construção guiada de uma aplicação completa em arquivo único (HTML + CSS + JS).

Código Fonte Completo

```
<!DOCTYPE html> <html lang="pt-BR"> <head> <meta charset="UTF-8">
<title>Contador</title> <style> body { font-family: sans-serif; display: flex; justify-
content: center; align-items: center; min-height: 100vh; background: #0f172a; margin: 0; }
.card { background: white; padding: 30px; border-radius: 12px; text-align: center; } #visor {
font-size: 64px; font-weight: bold; margin: 16px 0; color: #0f172a; } button { padding: 10px
16px; margin: 4px; border: none; border-radius: 6px; font-weight: bold; cursor: pointer; }
.btn-sub { background: #ef4444; color: white; } .btn-rst { background: #94a3b8; color: white;
} .btn-add { background: #10b981; color: white; } </style> </head> <body> <div class="card">
<h2>Contador Dinâmico</h2> <div id="visor">0</div> <button class="btn-sub" id="btnMenos">-
1</button> <button class="btn-rst" id="btnReset">Zerar</button> <button class="btn-add"
id="btnMais">+ 1</button> </div> <script> let contador = 0; // Estado da aplicação const
visor = document.querySelector("#visor"); const btnMais = document.querySelector("#btnMais");
const btnMenos = document.querySelector("#btnMenos"); const btnReset =
document.querySelector("#btnReset"); function renderizar() { visor.textContent = contador; if
(contador > 0) visor.style.color = "#10b981"; else if (contador < 0) visor.style.color =
"#ef4444"; else visor.style.color = "#0f172a"; } btnMais.addEventListener("click", () => {
contador++; renderizar(); }); btnMenos.addEventListener("click", () => { contador--;
renderizar(); }); btnReset.addEventListener("click", () => { contador = 0; renderizar(); });
</script> </body> </html>
```

Exercícios Extras de Extensão

Desafio 1: Trava de Limites

Impeça que o contador ultrapasse o intervalo entre -10 e +10, exibindo um alerta visual ou desabilitando o botão.

Desafio 2: Passo Customizado

Adicione um campo `<input type="number">` para permitir que o usuário escolha o valor de incremento (ex: pular de 5 em 5).